CS6600: Computer Architecture
Assignment-5: K-Means Clustering in GPU using CUDA

1 Problem Description

Let P be a list of N points in the plane, and let C be a list of K initial centroids. All coordinates are integers, and all indices are zero-based.

$$P = \{(x_0, y_0), \ldots, (x_{N-1}, y_{N-1})\}$$
$$C = \{(cx_0, cy_0), \ldots, (cx_{K-1}, cy_{K-1})\}$$

The task is to perform k-means clustering on P. Starting from the initial centroids C, each iteration has three steps: assign every point to its nearest centroid, accumulate the coordinate sums and point counts for each cluster, and replace each centroid by the integer-valued center computed from those totals. The iteration stops when an update leaves every centroid unchanged.

The algorithm returns the final cluster index of each point and the final coordinates of each centroid. Centroid indices remain fixed for the duration of the algorithm.

1.1 Assigning Clusters

The distance between a point (x, y) and a centroid (cx, cy) is the squared Euclidean distance

$$d\big((x, y), (cx, cy)\big) = (x - cx)^2 + (y - cy)^2.$$

Since $t \mapsto t^2$ is strictly increasing on $[0, \infty)$, the centroid that minimizes d is the centroid of minimum Euclidean distance.

Let cluster$[i]$ denote the index of the centroid to which point i is assigned:

$$\text{cluster}[i] = \arg \min_{k \in \{0,\ldots,K-1\}} d\big((x_i, y_i), (cx_k, cy_k)\big).$$

If two or more centroids attain this minimum, select the centroid of smallest index.

1.2 Accumulating Totals

For each cluster k, define the following totals. These totals are formed from the assignments of the current iteration and are the only inputs to the update step.

$$\text{sum}_x[k] = \sum_{i:\text{cluster}[i]=k} x_i, \qquad \text{sum}_y[k] = \sum_{i:\text{cluster}[i]=k} y_i,$$

$$c_k = |\{i : \text{cluster}[i] = k\}|.$$

1.3 Updating Centroids

If $c_k = 0$, the centroid (cx_k, cy_k) remains unchanged.

If $c_k \geq 1$, the updated coordinates are

$$cx_k = \left\lceil \frac{\text{sum}_x[k]}{c_k} - \frac{1}{2} \right\rceil, \qquad cy_k = \left\lceil \frac{\text{sum}_y[k]}{c_k} - \frac{1}{2} \right\rceil.$$

The expression rounds the quotient to the nearest integer. A quotient that lies exactly halfway between two integers is rounded to the smaller integer.

1.4 Algorithm Termination

At the end of an iteration, compare each updated centroid with the centroid of the same index used in the assignment step of that iteration. If every centroid is unchanged, the algorithm terminates, and the values cluster[i] from that assignment step are final. If any centroid has moved, replace the current centroids by the updated centroids and repeat the steps.

2 CUDA Essentials

CUDA is NVIDIA's programming model for general-purpose computation on a GPU. The CPU and its memory are referred to as the host, while the GPU and its memory are referred to as the device. Read the Programming Model and Intro to CUDA C++ chapters of the CUDA Programming Guide for the full account.

2.1 Declaring Kernels

A kernel is a function that runs on the device instead of the host. It is defined with the `__global__` specifier. When the host calls a kernel, it is executed by many threads in parallel.

```
__global__ void myKernel(int *d_array) {
    // GPU code goes here
}
```

2.2 Thread Hierarchy

Threads are organized in a hierarchy of grids, blocks, and threads. A grid consists of multiple blocks, and each block consists of multiple threads. Grids and blocks can be one-, two-, or three-dimensional. Each thread can determine its position using built-in indices. For a one-dimensional layout, the global index is

$$i = \texttt{blockIdx.x} \times \texttt{blockDim.x} + \texttt{threadIdx.x}.$$

2.3 Launching Kernels

A kernel launch corresponds to a grid and is initiated from the host using triple-chevron syntax. It specifies the grid size (number of blocks) and block size (number of threads per block). A block can contain up to 1024 threads, typically organized in multiples of 32.

```
int threadsPerBlock = 256;
int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;

myKernel<<<blocksPerGrid, threadsPerBlock>>>(d_array);
```

2.4 Memory Management

The typical workflow is to allocate device memory, copy data from the host to the device, run the kernel, and copy the results back. `cudaMalloc` allocates device memory, `cudaMemcpy` transfers data between the host and device, `cudaMemset` initializes device memory with a byte value, and `cudaFree` frees device memory.

```
int size = N * sizeof(int);
int *d_array;

cudaMalloc((void**)&d_array, size);
cudaMemset(d_array, 0, size);
cudaMemcpy(d_array, h_array, size, cudaMemcpyHostToDevice);

myKernel<<<blocks, threads>>>(d_array);

cudaMemcpy(h_array, d_array, size, cudaMemcpyDeviceToHost);
cudaFree(d_array);
```

3 Specification

3.1 Input and Output

The input file `input_xx.txt` contains whitespace-separated integers in the following order.

```
N K                             // number of points and centroids
x_0 x_1 ... x_{N-1}             // x-coordinates of the points
y_0 y_1 ... y_{N-1}             // y-coordinates of the points
cx_0 ... cx_{K-1}               // initial centroid x-coordinates
cy_0 ... cy_{K-1}               // initial centroid y-coordinates
```

The program writes its output to the path given as the second command-line argument. That output, and each reference file `expected_xx.txt`, must contain $K + 1$ lines in the following order.

```
cluster[0] ... cluster[N-1]  // cluster index of each point
cx_0 cy_0                    // final coordinates of centroid 0
cx_1 cy_1                    // final coordinates of centroid 1
...                          // remaining centroids, in index order
cx_{K-1} cy_{K-1}            // final coordinates of centroid K - 1
```

3.2 Example

The input file `input_00.txt` is

```
5 2
1 2 8 9 10
1 1 8 8 9
1 9
1 9
```

P is the list of points and C is the list of initial centroids.

$$P = \{(1,1),(2,1),(8,8),(9,8),(10,9)\} \qquad C = \{(1,1),(9,9)\}$$

In the first assignment step, d_k is the squared distance from a point to centroid k.

i	(x_i, y_i)	d_0	d_1	cluster[i]
0	$(1, 1)$	0	128	0
1	$(2, 1)$	1	113	0
2	$(8, 8)$	98	2	1
3	$(9, 8)$	113	1	1
4	$(10, 9)$	145	1	1

In the accumulation step, cluster 0 contains points 0 and 1, so $\text{sum}_x[0] = 3$, $\text{sum}_y[0] = 2$, and $c_0 = 2$. Cluster 1 contains points 2, 3, and 4, so $\text{sum}_x[1] = 27$, $\text{sum}_y[1] = 25$, and $c_1 = 3$.

In the update step,

$$cx_0 = \left\lceil \frac{3}{2} - \frac{1}{2} \right\rceil = 1, \qquad cy_0 = \left\lceil \frac{2}{2} - \frac{1}{2} \right\rceil = 1,$$

$$cx_1 = \left\lceil \frac{27}{3} - \frac{1}{2} \right\rceil = 9, \qquad cy_1 = \left\lceil \frac{25}{3} - \frac{1}{2} \right\rceil = 8.$$

The centroids are now $(1, 1)$ and $(9, 8)$.

In the second assignment step, the squared distances to these centroids are

i	(x_i, y_i)	d_0	d_1	cluster[i]
0	$(1, 1)$	0	113	0
1	$(2, 1)$	1	98	0
2	$(8, 8)$	98	1	1
3	$(9, 8)$	113	0	1
4	$(10, 9)$	145	2	1

The cluster indices are unchanged, so the accumulation step reproduces the same totals. The update step reproduces $(1, 1)$ and $(9, 8)$. Every centroid is unchanged, and the algorithm terminates.

The expected output file `expected_00.txt` is

```
0 0 1 1 1
1 1
9 8
```

3.3 Constraints

- $1 \leq N \leq 10^6$.
- $1 \leq K \leq 10^3$.
- $-10^3 \leq x, y \leq 10^3$.
- Every value computed by the algorithm fits in a signed 32-bit integer.
- Every input has one correct sequence of cluster indices and final centroids.

4 Implementation Guide

4.1 Deliverables

A `.zip` file is provided for the assignment and contains `handout.pdf`, `starter.cu`, and a `public_tests/` directory with `input_xx.txt` / `expected_xx.txt` pairs. Submit the completed `starter.cu` file, renamed to `<roll_number>.cu`, on Moodle.

4.2 Environment

This program can be built and run on Google Colab or on a computer with an NVIDIA GPU. To run it on Colab, open the website and create a new notebook. Under `Runtime` → `Change runtime type`, select `T4 GPU` as the hardware accelerator. Connect to the runtime, then upload the program and input files using the `Files` panel on the left. Run the commands below in a notebook cell to compile and execute the program.

```
!nvcc -O2 -std=c++17 <roll_number>.cu -o outbin
!./outbin input.txt output.txt
```

4.3 Strategy

The sequential loops identify the parallelism in k-means. Assignment examines each point independently. Accumulation adds each point's contribution independently, although several points may update the same cluster totals. The centroid update processes each centroid independently.

These independent pieces of work can run concurrently on the GPU. The host still coordinates each k-means iteration: it launches the GPU work in order and decides when the centroids have converged.

4.4 Notes

- The starter file already handles reading the input and writing the output. Complete only the parts marked by the comments.
- Rounding up the number of blocks may leave the last block with extra threads. A thread whose index is beyond the last element should return immediately.
- When multiple threads update the same memory location, a data race can occur. Use `atomicAdd(&sum, x)` in place of `sum += x`.
- Device memory is uninitialized after allocation. Initialize a buffer before using it to accumulate values.
- Operations issued in the default stream execute in order. A later kernel can use values written by an earlier kernel.